

Feb 21, 2025 7:45

IngredientTable.py

Page 1/5

```
1  # IngedientTable deals with the swiss food nutrition info
2  # the first iteration is english
3  #It uses pandas and will eventaully be called from ControDB
4
5  import csv
6  from io import StringIO
7
8  from sqlalchemy import create_engine, MetaData
9  import psycpg2
10 import pandas as pd
11 import numpy as np
12 import os
13 import sys
14 from loguru import logger
15
16 from pick_swiss_ingreds import Ui_Dialog
17
18 from PySide6.QtCore import (QSize, Qt ,QRect,
19 QCoreApplication,Slot,Signal,QAbstractListModel,QAbstractTableModel,QMetaObject,QModelIndex)
20 from PySide6.QtGui import QAction,QDoubleValidator,QFont
21
22 from PySide6.QtUiTools import QUiLoader
23
24 from PySide6.QtSql import QSqlDatabase , QSql,QSqlTableModel,QSqlQueryModel,QSqlQuery
25
26 from PySide6.QtWidgets import (
27 QApplication,
28 QCheckBox,
29 QComboBox,
30 QDateEdit,
31 QDateTimeEdit,
32 QDial,
33 QDialog,
34 QDoubleSpinBox,
35 QFileDialog,
36 QFontComboBox,
37 QFormLayout,
38 QHBoxLayout,
39 QLabel,
40 QLCDNumber,
41 QLineEdit,
42 ListView,
43 QMainWindow,
44 QWidget,
45 QProgressBar,
46 QPushButton,
47 QRadioButton,
48 QSlider,
49 QSpinBox,
50 QStatusBar,
51 QTimeEdit,
52 QTableView,
53 QMenuBar,
54 QVBoxLayout,
55 QAbstractItemView,
56 QGridLayout,
57 QTextEdit
```

Feb 21, 2025 7:45

IngredientTable.py

Page 2/5

```

58 )
59 )
60
61 class IngredDialog(QDialog,Ui_Dialog):
62     def __init__(self):
63         super().__init__()
64         self.setupUi(self)
65
66
67
68 # here we setup the model
69
70 class IngredModel(QAbstractTableModel):
71     def __init__(self, ingredients):
72         super().__init__()
73         self.ingredients = ingredients
74
75     def data(self, index, role):
76         if role == Qt.DisplayRole:
77             value = self.ingredients.iloc[index.row(), index.column()]
78             return str(value)
79
80     def rowCount(self, index):
81         return self.ingredients.shape[0]
82
83     def columnCount(self, index):
84         return self.ingredients.shape[1]
85
86     def headerData(self, section, orientation, role):
87         if role == Qt.DisplayRole:
88             if orientation == Qt.Horizontal:
89                 return str(self.ingredients.columns[section])
90
91             if orientation == Qt.Vertical:
92                 return str(self.ingredients.index[section])
93
94
95
96
97 class IngredientTable(QMainWindow):
98     def __init__(self,table_name = None):
99         super().__init__()
100
101         self.setup_logger()
102
103         self.read_table(table_name)
104
105
106         self.INTview = QTableView()
107
108         ingredients = self.df
109
110
111         self.IngredModel = IngredModel(ingredients)
112         self.INTview.setModel(self.IngredModel)
113
114         #if I click on a cell go to connect

```

Feb 21, 2025 7:45

IngredientTable.py

Page 3/5

```

115         self.INTview.clicked.connect(self.print_cell)
116
117         #Set geometry of table_view
118         self.mod_table_display()
119
120         self.setCentralWidget(self.INTview)
121         self.setGeometry(20, 20, 1200, 400)
122
123
124
125     def find_pattern(self, word=None):
126         """this routine goes through the names in the database
127         and returns a list of values which will then go into a combo_box dialog to choose from
128         the names are in cloumn 0
129         returns the list of matches together with how many times it found a match
130         count =1 for exactly one match.
131         """
132         self.pattern_count=0 # how many times the pattern was found
133
134         self.myPatternFound = []
135         temp = self.df['name'].to_list()
136         # we check for the first three characters case insensitive
137         for k in range(len(temp)):
138             needle = "TEST"
139             if word[0:4].casefold() in temp[k][0:4].casefold():
140                 self.myPatternFound.append(temp[k])
141                 self.pattern_count = self.pattern_count +1
142
143         return
144
145
146     def get_ingredient(self):
147
148         self.GI = IngredDialog()
149
150         #populate the values
151
152         self.GI.comboPick.insertItems(0,self.myPatternFound)
153
154         # self.GI.comboPick.textHighlighted.connect(self.print_selection)
155         self.GI.comboPick.currentTextChanged.connect(self.get_ingred_selection)
156
157         self.GI.show()
158         return
159
160     def get_ingred_selection(self, s):
161
162         self.selected_ingredient = s
163         logger.debug("you selected %s" % self.selected_ingredient)
164
165
166
167
168
169
170
171

```

Feb 21, 2025 7:45

IngredientTable.py

Page 4/5

```

172
173
174
175     def mod_table_display(self):
176         """modify how the table looks"""
177         self.INTview.resizeColumnsToContents()
178
179     def print_cell(self, index):
180
181         #all data communication happes through self.IngredModel.ingredients
182         row = index.row()
183         column = index.column()
184         data = index.data()
185
186         logger.debug("you selected %s " %self.IngredModel.ingredients.iloc[index.row(), index.column()])
187
188     def read_table(self, table = None):
189         """reads the csv ingredients file"""
190
191         self.df = pd.read_csv(table)
192
193
194
195     def setup_logger(self):
196
197         #logger.remove(0)
198         #now we add color to the terminal output
199         logger.add(sys.stdout,
200                     colorize = True, format="<green>{time}</green> {function} {line} {level} <level>{message}</level>" ,
201                     level = "DEBUG")
202
203
204
205         fmt = "{time} - {name} - {function} - {line} - {level} - {message}"
206         logger.add('info.log', format = fmt , level = 'INFO', rotation="1 day")
207
208
209         # set the colors of the different levels
210         logger.level("INFO", color = '<black>')
211         logger.level("WARNING", color = '<green>')
212         logger.level("ERROR", color = '<red>')
213         logger.level("DEBUG", color = '<blue>')
214
215     return
216
217     def do_sql_table(self):
218         conn_string = 'postgres://klein:xxxx@192.168.2.164:5432/recipe_ak'
219         engine = create_engine(conn_string)
220         conn = engine.connect()
221         self.df.to_sql('swiss_food', con=conn, if_exists= "replace" , index = False)
222
223         conn = psycopg2.connect(conn_string
224                                 )
225         conn.autocommit = True
226         cursor = conn.cursor()
227
228         sql1 = '''select * from swiss_food;'''

```

Feb 21, 2025 7:45

IngredientTable.py

Page 5/5

```

229         cursor.execute(sql1)
230         for i in cursor.fetchall():
231             print(i)
232
233         # conn.commit()
234         conn.close()
235
236
237     def psql_insert_copy(table, conn, keys, data_iter):
238         # gets a DBAPI connection that can provide a cursor
239         dbapi_conn = conn.connection
240         with dbapi_conn.cursor() as cur:
241             s_buf = StringIO()
242             writer = csv.writer(s_buf)
243             writer.writerows(data_iter)
244             s_buf.seek(0)
245
246             columns = ','.join('"%{}"'.format(k) for k in keys)
247             if table.schema:
248                 table_name = '{}.{}'.format(table.schema, table.name)
249             else:
250                 table_name = table.name
251
252             sql = 'COPY {} (()) FROM STDIN WITH CSV'.format(
253                 table_name, columns)
254             cur.copy_expert(sql=sql, file=s_buf)
255
256
257
258 if __name__ == '__main__':
259
260     app = QApplication(sys.argv)
261
262
263
264
265
266     table_name = '/Users/klein/git/qt_exercises/nutrition_databases/swiss_food.csv'
267
268     InTa = IngredientTable(table_name=table_name)
269     InTa.do_sql_table()
270     InTa.find_pattern(word='Almond')
271     InTa.get_ingredient()
272     InTa.show()
273
274
275     app.exec()
```